

Chapitre 1

La compilation par l'exemple

Notes rédigées par Raphaël JONTEF

cours dispensé par M. Janin

Enseirb-Matmeca
filière informatique

7 septembre 2026

Table des matières

I	Analyse lexicale	2
II	Analyse syntaxique	2
III	Analyse sémantique	2
IV	Notion de passe de compilation	3
V	Phases aval de compilation	3

Il existe trois couches principales dans un compilateur :

- l'analyse lexicale : elle consiste à transformer une suite de caractères en une suite de mots. Cette phase est réalisée par un analyseur lexical.
- l'analyse syntaxique : elle consiste à vérifier que la suite de mots obtenue par l'analyse lexicale respecte les règles de grammaire du langage. Cette phase est réalisée par un analyseur syntaxique.
- l'analyse sémantique : elle consiste à vérifier que le programme est sémantiquement correct, c'est-à-dire qu'il respecte les règles de validité des opérations et des identificateurs. Cette phase est réalisée par un analyseur sémantique.

I Analyse lexicale

L'analyse lexicale consiste en la conversion de l'une suite de caractères en une suite de mots.

Exemple 1.1

p	=	p0	+	v	*	t
ID	EQ	ID	PLUS	ID	TIMES	ID

Pour parvenir à cela, l'analyseur lexical va utiliser un ensemble de règles de reconnaissance de mots, appelées *expressions régulières*. Ces expressions régulières sont des motifs qui décrivent les séquences de caractères qui forment des mots valides dans le langage. On utilise des analyseurs lexicaux comme `flex` ou `lex` pour générer automatiquement le code de l'analyseur lexical à partir de ces expressions régulières.

II Analyse syntaxique

Pour analyser la syntaxe de la suite de mots obtenue par l'analyse lexicale, on utilise un analyseur syntaxique. L'analyse syntaxique consiste à vérifier si la suite de mots respecte les règles de grammaire du langage. Cette grammaire spécifie donc la syntaxe (priorités de calculs, associativité, etc.) du langage. L'analyse syntaxique est souvent réalisée à l'aide d'arbres syntaxiques abstraits (AST) qui représentent la structure hiérarchique de la suite de mots.

On utilise des analyseurs syntaxiques comme `bison` ou `yacc` pour générer automatiquement le code de l'analyseur syntaxique à partir de la grammaire du langage.

III Analyse sémantique

Après l'analyse syntaxique, on obtient une représentation structurée du programme, sous la forme d'un arbre syntaxique abstrait (AST). Cependant, cette représentation ne garantit pas que le programme est sémantiquement correct.

Exemple 1.2

Le programme peut multiplier un entier par un flottant, ce qui est une opération soit à invalider, soit à convertir implicitement. L'analyse sémantique va donc vérifier la validité des opérations et effectuer les conversions nécessaires.

Il y a donc au cours de cette phase de l'analyse sémantique, deux tâches principales :

- analyse de type : vérifier que les opérations sont valides pour les types d'opérandes donnés, et effectuer les conversions nécessaires si possible.
- analyse de noms : vérifier que les identificateurs utilisés dans le programme sont déclarés et accessibles dans le contexte actuel.

IV Notion de passe de compilation

Les phases de compilation ne sont pas à confondre avec les *passes de compilation*. Une passe de compilation est un passage entier sur le code source qui peut effectuer plusieurs phases de compilation. Par exemple, une passe de compilation peut effectuer la phase d'analyse lexicale et la phase d'analyse syntaxique. Il faudra alors réaliser au moins deux passes de compilation pour effectuer toutes les phases de compilation.

Nous ferons en particulier de la production de code à la volée

V Phases aval de compilation

- **Production de code intermédiaire** : le code intermédiaire est un code qui n'est pas encore du code machine mais qui est plus proche de la machine que le code source. Il est souvent utilisé pour faciliter l'optimisation du code et pour permettre la portabilité du compilateur sur différentes architectures.
- **Optimisation du code** : cette phase consiste à améliorer le code intermédiaire pour le rendre plus efficace en termes de temps d'exécution et de consommation de mémoire. Les optimisations peuvent inclure la suppression de code mort, la réorganisation des instructions, l'inlining de fonctions, etc.
- **Production de code machine ou code cible** : cette phase consiste à traduire le code intermédiaire optimisé en code machine spécifique à l'architecture cible. Le code machine est le code exécutable par le processeur de l'ordinateur.

On va utiliser les grammaires hors contexte.

Exemple 1.3 - *fil rouge*

Sur l'alphabet d'entrée $T = \{ (,), +, *, \text{id}, \text{cot} \}$, on souhaite définir formellement les expressions arithmétiques bien formées.

Soit $E \subset T^*$ ledit langage. On a :

$$\text{id} \in E$$

$$E + E \subset E$$

$$E * E \subset E$$

$$(E) \subset E$$

Nous appellerons $\mathcal{F}(E)$ l'union de ces parties distinctes. L'ensemble recherché est alors :

$$E = \bigcup_{n \in \mathbb{N}} \mathcal{F}^n(\emptyset)$$